

DATA/CS 172: Python for Data Analysis

Dictionaries as Counters

Prof. Travis Mandel

10/9/2024



UNIVERSITY
of HAWAII®

HILO

Reminders

- Program 1 due Friday!
- Stay on top of Labs

Dictionaries

- Empty dict
 - `myDict = {}`
 - Note the curly braces!
 - In contrast to square for lists and parens for tuples

- Dict with some stuff in there already:

```
myDict = {  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3"  
}
```

Keys often strings

Values could be any type (int, float, string, etc.)

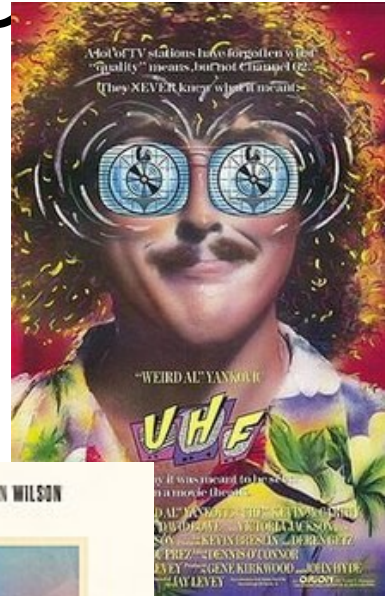
Common use case: Counting stuff



- Trying to count how many things of a certain type are in our data
- Example: How many films is an actor in?
 - How many times does a certain sensor reading appear?
 - Etc.
- Solution: Use a dict
 - Keys are strings etc.
 - Values are **ints**
 - Values represent counts
 - Start at 0, go up by 1 as we see items

Great! However, in practice we will come upon this problem

- actorCount = {"Nick Cage":112, "Tom Cruise":48}
- actorCount["Nick Cage"] += 1
 - #actorCount["Nick Cage"] is now 113
- # We see a new actor, "weird al", in our dataset
- actorCount["weird al"] +=1
 - Crashes our code with a KeyError!!!
 - Demo



One way to deal with this...

- If “weird al” not in actorCount:
 - actorCount[“weird al”] = 0
- actorCount[“weird al”] += 1
- Issues
 - Still have a problem when reporting counts of items that don’t exist
 - An if statement seems a bit extreme for something that is so common
 - Conceptually, just adding one, why do I need multiple lines?
 - Lookup happens up to THREE times instead of once
 - This can slow down the code!



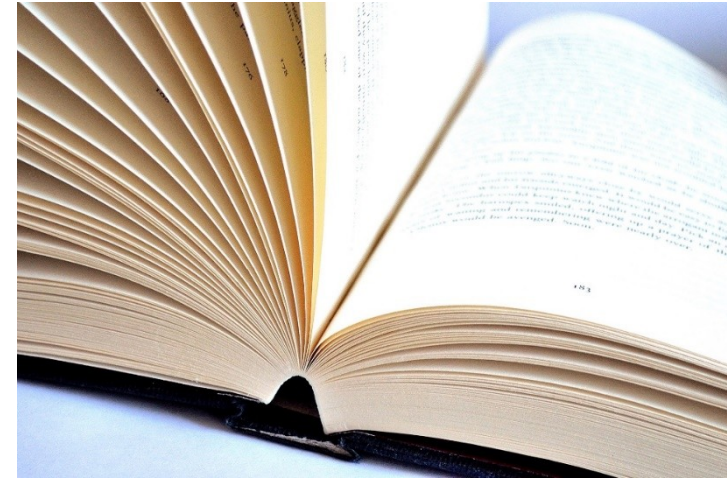
Another (MUCH BETTER) way to deal with missing keys

- `myDict.get(key, defaultVal)`
- If key is not present, returns `defaultVal`
- For counting dicts, often useful to do `myDict.get(key, 0)`
 - Count should be zero if key not in dict
- So when adding one it would be:
 - `myDict[key] = myDict.get(key, 0) + 1`
- **When working with integer values, and you have a default value, please use `.get()`!!**



One place where this can be particularly useful...

- Working with text data!
 - Natural Language Processing
 - Text Mining
 - Computational Linguistics
- Often given a corpus
 - A corpus (plural: corpora) is a large amount of text data suitable for linguistic analysis
 - For instance, the famous and widely-used Brown corpus
 - Collected in 1961
 - About 1 million words
 - Contains mostly scholarly writings such as news articles, books, etc.
- Have to identify and count patterns that occur in the corpus
 - For example, how common is a certain word or phrase compared to another?
 - Google has always been pretty terrible at this kind of thing...
 - ChatGPT does a (much) more sophisticated version of this to “learn” what sequences of words are more likely than others



Lab Out

- Counting words in the Brown corpus